

L02 · Ασυμπτωτική Ανάλυση

Algorithms Class Hub

Αλγόριθμοι και Πολυπλοκότητα (Κ17) · ΕΚΠΑ

Η γλώσσα με την οποία συγκρίνουμε αλγόριθμους: O , Θ , Ω , o , ω , η ιεραρχία πολυπλοκοτήτων, και η οριακή προσέγγιση με $L'Hôpital$.

1 Η ερώτηση που θέλουμε να απαντήσουμε

Πώς επηρεάζεται η απόδοση ενός αλγόριθμου όταν ο όγκος των δεδομένων που λαμβάνει αυξάνεται;

Στο πρώτο μάθημα είπαμε ότι η πολυπλοκότητα εκφράζεται ως συνάρτηση της διάστασης n . Εδώ θα μάθουμε να συγκρίνουμε τέτοιες συναρτήσεις χωρίς να μας ενδιαφέρουν σταθεροί συντελεστές ή όροι χαμηλότερης τάξης — γιατί όταν το n μεγαλώνει, μόνο ο **κυρίαρχος όρος** μετράει.

Διαίσθηση

Σκέψου ότι κρίνεις δύο φοιτητές για τα μαθηματικά: ο ένας λύνει σύνθετα προβλήματα σε 5 λεπτά, ο άλλος σε 5 λεπτά και 30 δευτερόλεπτα. Αν όλη η ζωή ήταν μαθηματικά, η διαφορά θα ήταν ασήμαντη. Αλλά αν ο πρώτος λύνει σε 5 λεπτά και ο δεύτερος σε **2 ώρες**, ο πρώτος είναι ασύγκριτα καλύτερος — όχι κατά λίγο, αλλά σε διαφορετική κλίμακα. Η ασυμπτωτική ανάλυση μετρά ακριβώς αυτή τη διαφορά κλίμακας.

2 Γιατί δεν αρκούν οι σταθερές

Παράδειγμα

Δύο αλγόριθμοι για το ίδιο πρόβλημα.

- A_1 τρέχει σε $T_1(n) = 100n$ στοιχειώδεις πράξεις.
- A_2 τρέχει σε $T_2(n) = n^2$.

Για ποιο n είναι καλύτερος ο καθένας; Λύνουμε $100n = n^2 \Rightarrow n = 100$. Άρα:

- Για $n < 100$ ο A_2 είναι ταχύτερος.
- Για $n = 100$ είναι ισοδύναμοι.
- Για $n > 100$ ο A_1 είναι ταχύτερος — και η διαφορά **μεγαλώνει συνεχώς**.

Αν θέσουμε ότι μία στοιχειώδης πράξη παίρνει 10^{-6} sec:

n	$A_1 (100n)$	$A_2 (n^2)$	Λόγος A_2/A_1
1.000	0.1 sec	1 sec	$10\times$
100.000	10 sec	10.000 sec $\approx$ 2.8 ώρες	$1.000\times$

Παρά τη μεγάλη σταθερά (100) στον A_1 , για μεγάλο n ο A_1 νικάει σε ακαριαία βάση. **Η σταθερά δεν σώζει έναν αλγόριθμο με χειρότερη τάξη μεγέθους.**

Αυτό το παράδειγμα είναι ο λόγος που η ασυμπτωτική ανάλυση **αγνοεί** σταθερές. Αν η μία μέτρηση είναι $100n$ και η άλλη n^2 , η σταθερά 100 είναι μικρή σε σχέση με τη διαφορά «γραμμικό vs τετραγωνικό».

3 Οπτική σύγκριση συναρτήσεων

Φαντάσου ότι σχεδιάζουμε στον ίδιο άξονα τις κλασικές κλάσεις πολυπλοκότητας — $\log n$, n , $n \log n$, n^2 και 2^n . Παρατήρησε πώς η εκθετική 2^n ξεπερνά κάθε λογικό όριο για $n \approx 13$, ενώ η λογαριθμική κάμπτεται σχεδόν αμέσως και μένει σχεδόν επίπεδη. Αυτή η οπτική διαφορά κλίμακας είναι όλο το παιχνίδι.

4 Οι τυπικοί ορισμοί

Έστω $f, T : \mathbb{N} \rightarrow \mathbb{R}^+$. Ορίζουμε **τρεις κλάσεις συναρτήσεων** σε σχέση με την f :

$$\begin{aligned} O(f(n)) &= \{T : \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \geq n_0, T(n) \leq c \cdot f(n)\} \\ \Omega(f(n)) &= \{T : \exists n_0 \in \mathbb{N}, \exists c \in \mathbb{R}^+, \forall n \geq n_0, T(n) \geq c \cdot f(n)\} \\ \Theta(f(n)) &= O(f(n)) \cap \Omega(f(n)) \end{aligned}$$

Σε απλά Ελληνικά:

- $T(n) = O(f(n))$ — η T «δεν αυξάνεται γρηγορότερα» από την f . Άνω φράγμα.
- $T(n) = \Omega(f(n))$ — η T «δεν αυξάνεται πιο αργά» από την f . Κάτω φράγμα.
- $T(n) = \Theta(f(n))$ — η T αυξάνεται **στον ίδιο ρυθμό** με την f (μέχρι σταθεράς).

Κλειδί

Σταθερά + n_0 είναι ο πυρήνας του ορισμού. Η συνθήκη ισχύει «τελικά» — από κάποιο σημείο n_0 και μετά — και «έως σταθεράς c ». Έτσι σταθερές και αρχικές διαφορές δεν μετράνε. Μετράει μόνο η **ασυμπτωτική** συμπεριφορά.

5 Παραδείγματα ορισμού

Παράδειγμα

Πρόβλημα. Έστω $T(n) = 3n^2 - 100n + 6$. Δείξε ότι:

1. $T(n) \in O(n^2)$ — γιατί $3n^2 - 100n + 6 < 3n^2$ για κάθε $n \geq 1$. (Πάρε $c = 3$, $n_0 = 1$.)
2. $T(n) \in O(n^3)$ — για κάθε σταθερά $c > 0$, ισχύει $3n^2 - 100n + 6 < c \cdot n^3$ για αρκετά μεγάλο

n . Δηλαδή το $O(n^3)$ ισχύει αλλά είναι «ξεκάθαρα ασθενέστερο» από το $O(n^2)$.

3. $T(n) \notin O(n)$ — γιατί για κάθε σταθερά c , ισχύει $c \cdot n < 3n^2$ για $n > c$. Άρα ο T ξεπερνά τελικά κάθε γραμμική σταθερά.

Μάθημα: όταν λέμε « $T(n) = O(g(n))$ » δεν λέμε την ακριβή τάξη, λέμε ένα άνω φράγμα. Το $O(n^3)$ ισχύει αλλά είναι χαλαρό. Στο εξεταστικό ζητάμε πάντα το «καλύτερο» (πιο σφιχτό) O , που συνήθως είναι το Θ .

6 Η ιεραρχία πολυπλοκοτήτων

Σε αυξανόμενη σειρά ρυθμού αύξησης, οι κλασικές κλάσεις είναι:

$$\underbrace{1 \prec \log n \prec n \prec n \log n \prec n^2}_{\text{πολυωνυμικό (καλό)}} \prec \underbrace{2^n \prec n!}_{\text{εκθετικό (κακό)}}$$

Κάθε κλάση «κυριαρχείται» από την επόμενη: για κάθε διαδοχικό ζευγάρι, η αριστερή ανήκει στο $o(\text{δεξιά})$.

Κλειδί

Ο πρακτικός κανόνας του μαθήματος:

- Πολυωνυμική πολυπλοκότητα (n^c για σταθερά c) → καλή.
- Εκθετική (a^n για $a > 1$) → κακή.

Η διαχωριστική γραμμή είναι αυτή. Όλοι οι αλγόριθμοι που θα δούμε στοχεύουν πολυωνυμική πολυπλοκότητα.

7 Wheat and chessboard — γιατί το εκθετικό είναι «καταστροφή»

Παράδειγμα

Ο μύθος. Λέγεται ότι ο εφευρέτης του σκακιού ζήτησε ως αμοιβή από τον βασιλιά: 1 κόκκο σιταριού στο πρώτο τετράγωνο, 2 στο δεύτερο, 4 στο τρίτο, ...διπλασιάζοντας μέχρι το 64ο τετράγωνο. Ο βασιλιάς, που νόμιζε ότι το αίτημα ήταν ασήμαντο, δέχτηκε. Το συνολικό σιτάρι:

$$T_{64} = 1 + 2 + 4 + \dots + 2^{63} = 2^{64} - 1 \approx 1.8 \times 10^{19}$$

Δηλαδή **18 δισ. δισ. κόκκοι** — περισσότερο σιτάρι από όσο έχει παραχθεί ποτέ στην ιστορία της ανθρωπότητας. Ο βασιλιάς δεν είχε τρόπο να πληρώσει.

Αυτό είναι το 2^n σε δράση. Αν ένας αλγόριθμος είναι $O(2^n)$, αρκεί να αυξήσεις το n κατά 1 για να διπλασιαστεί ο χρόνος εκτέλεσης.

8 Η «πικρή πραγματικότητα»: γιατί δεν μας σώζουν οι ταχύτεροι υπολογιστές

Μια έντονη παρανόηση είναι ότι «οι αλγόριθμοι δεν έχουν τόση σημασία γιατί οι υπολογιστές γίνονται όλο και ταχύτεροι». Ας το ελέγξουμε: **έστω υπολογιστής 1000 φορές ταχύτερος από τον σημερινό.** Πόσο μεγαλύτερο πρόβλημα μπορούμε να λύσουμε στον ίδιο χρόνο;

Πολυπλοκότητα	Τωρινό μέγεθος	Μελλοντικό μέγεθος ($\times 1000$ ταχύτητα)
$O(n)$	n	$1000n$
$O(n^2)$	n	$n \cdot \sqrt{1000} \approx 31.6n$
$O(n^3)$	n	$n \cdot 1000^{1/3} \approx 10n$
$O(n^4)$	n	$n \cdot 1000^{1/4} \approx 5.6n$
$O(2^n)$	n	$n + \log_2 1000 \approx n + 10$
$O(3^n)$	n	$n + \log_3 1000 \approx n + 6.3$
$O(n!)$	n	$\approx n$ (ουσιαστικά αμετάβλητο)

Προσοχή

Με $1000\times$ ταχύτερο μηχάνημα, ένας **γραμμικός** αλγόριθμος λύνει 1000 φορές μεγαλύτερο πρόβλημα. Ένας **εκθετικός** αλγόριθμος λύνει πρόβλημα **10 παραπάνω εισόδων**. Καμία υλική ταχύτητα δεν σώζει έναν εκθετικό αλγόριθμο. Γι' αυτό μετράει ο αλγόριθμος, όχι το μηχάνημα.

9 Ιδιότητες των τάξεων μεγέθους

Έστω $f, f_1, f_2 : \mathbb{N} \rightarrow \mathbb{R}^+$. Όλα τα παρακάτω αποδεικνύονται κατευθείαν από τον ορισμό:

1. $f(n) \in O(f(n))$ — κάθε συνάρτηση «κυριαρχείται από τον εαυτό της».
2. $f(n) \in \Theta(f(n))$ — και «είναι ίσης τάξης με τον εαυτό της».
3. $O(c \cdot f(n)) = O(f(n))$ — οι σταθεροί συντελεστές απορροφώνται.
4. $O(f(n) + f(n)) = O(f(n))$ — διπλασιάζοντας μια συνάρτηση δεν αλλάζει τάξη.
5. $O(f_1(n) + f_2(n)) = O(\max\{f_1(n), f_2(n)\})$ — σε άθροισμα μετράει ο κυρίαρχος όρος.
6. $O(f_1(n)) \cdot O(f_2(n)) = O(f_1(n) \cdot f_2(n))$ — οι τάξεις πολλαπλασιάζονται.
7. $f(n) \in O(g(n)) \iff g(n) \in \Omega(f(n))$ — O και Ω είναι δυϊκά.
8. $f(n) \in \Theta(g(n)) \iff f(n) \in O(g(n)) \text{ ΚΑΙ } g(n) \in O(f(n))$ — το Θ θέλει και τις δύο κατευθύνσεις.
9. $f(n) \in \Theta(g(n)) \iff g(n) \in \Theta(f(n))$ — το Θ είναι συμμετρικό.
10. $f(n) \in \Theta(g(n))$ και $g(n) \in \Theta(h(n)) \Rightarrow f(n) \in \Theta(h(n))$ — μεταβατικότητα.

Παράδειγμα

Απόδειξη της 5ης ιδιότητας ($O(f_1 + f_2) = O(\max\{f_1, f_2\})$).

Έστω $g(n) \in O(f_1(n))$ και $g(n) \in O(f_2(n))$. Από τον ορισμό:

$$\exists c_1, n_1 : \forall n \geq n_1, g(n) \leq c_1 f_1(n)$$

$$\exists c_2, n_2 : \forall n \geq n_2, g(n) \leq c_2 f_2(n)$$

Πάρε $c_0 = \max\{c_1, c_2\}$, $n_0 = \max\{n_1, n_2\}$. Τότε για κάθε $n \geq n_0$:

$$g(n) \leq c_0(f_1(n) + f_2(n)) \leq 2c_0 \max\{f_1(n), f_2(n)\}$$

Άρα $g(n) \in O(\max\{f_1(n), f_2(n)\})$. ■

10 Είναι κάθε δύο συναρτήσεις ασυμπτωτικά συγκρίσιμες;

Όχι. Υπάρχουν συναρτήσεις που η μία δεν είναι ούτε O ούτε Ω της άλλης.

Παράδειγμα

Αντιπαράδειγμα. Έστω $f(n) = n$ και $g(n) = n^{1+\sin(n)}$. Ο εκθέτης $1 + \sin(n)$ ταλαντώνεται μεταξύ 0 και 2:

- Όταν $\sin(n) \approx 1$, η $g(n) \approx n^2$ (αυξάνεται πολύ πιο γρήγορα από f).
- Όταν $\sin(n) \approx -1$, η $g(n) \approx n^0 = 1$ (αυξάνεται πολύ πιο αργά από f).

Άρα δεν μπορούμε να πούμε ούτε «η g είναι $O(n)$ » ούτε «η g είναι $\Omega(n)$ ». **Αυτές οι δύο συναρτήσεις είναι ασυμπτωτικά ασύγκριτες.**

11 Πρακτικές ασκήσεις

Παράδειγμα

Άσκηση 1. Ναδειχθεί ότι αν $a > 1$ και $f(n) \in O(\log_a n)$, τότε $f(n) \in O(\log_2 n)$. (Με άλλα λόγια: η βάση του λογάριθμου δεν παίζει ρόλο στο O .)

Λύση. Από τον ορισμό: $f(n) \leq c \log_a n$ για κάποια σταθερά c . Με αλλαγή βάσης: $\log_a n = \frac{\log_2 n}{\log_2 a}$.

Άρα:

$$f(n) \leq c \cdot \frac{\log_2 n}{\log_2 a} = c' \cdot \log_2 n$$

όπου $c' = c / \log_2 a$ είναι σταθερά (γιατί $a > 1$ δίνει $\log_2 a > 0$). Άρα $f(n) \in O(\log_2 n)$. ■

Συμπέρασμα. Γράφουμε απλά $O(\log n)$ — δεν χρειάζεται να γράψουμε τη βάση.

Παράδειγμα

Άσκηση 2. Ναδειχθεί ότι $\log(n!) \in O(n \log n)$.

Λύση. Παρατήρησε:

$$n! = 1 \cdot 2 \cdot 3 \cdots n \leq n \cdot n \cdot n \cdots n = n^n$$

Αφού η λογαριθμική συνάρτηση είναι αύξουσα:

$$\log(n!) \leq \log(n^n) = n \log n$$

Άρα $\log(n!) \in O(n \log n)$ με σταθερά $c = 1$. ■

Bonus. Η αντίστροφη κατεύθυνση ισχύει επίσης ($\log(n!) \in \Omega(n \log n)$), οπότε $\log(n!) \in \Theta(n \log n)$ — αυτό θα το χρειαστούμε όταν αποδείξουμε ότι η ταξινόμηση με συγκρίσεις απαιτεί τουλάχιστον $\Omega(n \log n)$ συγκρίσεις (κάτω φράγμα).

Παράδειγμα

Άσκηση 3. Βρες τον καλύτερο (πιο σφιχτό) O -συμβολισμό για κάθε έκφραση:

(α) $2 \log n - 4n + 3n \log n$. Ο κυρίαρχος όρος είναι ο $3n \log n$ (κάθε άλλος είναι χαμηλότερης τάξης). Άρα $O(n \log n)$ — στην πραγματικότητα και $\Theta(n \log n)$.

(β) $2 + 4 + \dots + 2n = 2(1 + 2 + \dots + n) = 2 \cdot \frac{n(n+1)}{2} = n^2 + n$. Άρα $\Theta(n^2)$.

(γ) $2 + 4 + 8 + \dots + 2^n = 2^{n+1} - 2$. Άρα $\Theta(2^n)$ — η ταυτότητα της γεωμετρικής σειράς δίνει ακριβές αποτέλεσμα.

Παράδειγμα

Άσκηση 4. Δείξε ότι:

(α) $\sum_{k=1}^n k \cdot n \in O(n^3)$. Πράγματι $\sum_{k=1}^n k \cdot n = n \sum_{k=1}^n k = n \cdot \frac{n(n+1)}{2} = \frac{n^2(n+1)}{2} \leq n^3$. Άρα $O(n^3)$.

(β) Αν $f(n) = O(n)$, τότε $(f(n))^2 = O(n^2)$. Αν $f(n) \leq c \cdot n$, τότε $(f(n))^2 \leq c^2 \cdot n^2$. Άρα $(f(n))^2 = O(n^2)$.

(γ) $3^n \notin O(2^n)$. Λόγος: $\frac{3^n}{2^n} = (3/2)^n \rightarrow \infty$. Άρα δεν υπάρχει σταθερά c με $3^n \leq c \cdot 2^n$ για κάθε $n \geq n_0$ — το 3^n ξεπερνά τελικά κάθε σταθερό πολλαπλάσιο του 2^n .

12 Εναλλακτική προσέγγιση: οριακές σχέσεις

Αντί να δείξουμε τυπικά «υπάρχει c και n_0 », μπορούμε να χρησιμοποιήσουμε **όρια**. Έστω $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \alpha \neq 0 \Rightarrow f \in \Theta(g)$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \Rightarrow f \in O(g) \text{ και } f \notin \Theta(g)$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \Rightarrow g \in O(f) \text{ και } g \notin \Theta(f)$$

Οι μεσαίες περιπτώσεις (όριο 0 ή ∞) δίνουν τους «αυστηρά μικρότερα» συμβολισμούς:

- $\lim \rightarrow 0 \rightarrow$ γράφουμε $f(n) \in o(g(n))$ («μικρό-ο»). Διαβάζεται «η f είναι **αυστηρά** μικρότερης τάξης από τη g ».
- $\lim \rightarrow \infty \rightarrow$ γράφουμε $f(n) \in \omega(g(n))$ («μικρό-ωμέγα»). Διαβάζεται «η f είναι **αυστηρά** μεγαλύτερης τάξης».

Πιο τυπικά:

$$o(f(n)) = \{g : \forall c > 0, \exists n_0, \forall n \geq n_0, g(n) < c \cdot f(n)\}$$

$$\omega(f(n)) = \{g : \forall c > 0, \exists n_0, \forall n \geq n_0, g(n) > c \cdot f(n)\}$$

Κλειδί

Η διαφορά O vs o :

- $g \in O(f) \rightarrow$ υπάρχει κάποιος c με $g(n) \leq cf(n)$ τελικά. Η g ίσως είναι «του ίδιου ρυθμού».
- $g \in o(f) \rightarrow$ για κάθε c μικρό, ισχύει $g(n) < cf(n)$ τελικά. Η g είναι **αυστηρά** πιο αργή.

Το Θ αποκλείεται με o . Το Θ επιτρέπεται με O .

13 Υπενθύμιση — Κανόνας του L'Hôpital

Πολλά από τα όρια που χρειαζόμαστε δίνουν απροσδιοριστία ∞/∞ ή $0/0$. Σε αυτές τις περιπτώσεις:

Αν $\lim_{x \rightarrow \infty} f(x) = +\infty$ και $\lim_{x \rightarrow \infty} g(x) = +\infty$, και οι f, g είναι παραγωγίσιμες, τότε:

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{f'(x)}{g'(x)}$$

Όταν η απροσδιοριστία επιμένει, εφαρμόζουμε ξανά. Το χρησιμοποιούμε αμέσως παρακάτω.

14 Δύο σημαντικά αποτελέσματα για τους ρυθμούς αύξησης

14.1 Εκθετικό κυριαρχεί πάνω σε πολυωνυμικό

Κάθε εκθετική συνάρτηση με βάση μεγαλύτερη του 1 αυξάνεται γρηγορότερα από κάθε πολυωνυμική συνάρτηση.

Τυπικά, για κάθε $a > 1$ και $b \in \mathbb{R}$:

$$\lim_{n \rightarrow \infty} \frac{n^b}{a^n} = 0 \quad \Leftrightarrow \quad n^b \in o(a^n)$$

Παράδειγμα

Απόδειξη. Εφαρμόζουμε L'Hôpital επανειλημμένα:

$$\lim_{x \rightarrow \infty} \frac{x^b}{a^x} = \lim_{x \rightarrow \infty} \frac{b \cdot x^{b-1}}{a^x \ln a} = \lim_{x \rightarrow \infty} \frac{b(b-1)x^{b-2}}{a^x \ln^2 a} = \dots = \lim_{x \rightarrow \infty} \frac{b!}{a^x \ln^b a} = 0$$

Μετά από $[b]$ εφαρμογές του L'Hôpital, ο αριθμητής γίνεται σταθερά και ο παρονομαστής εξακολουθεί να αυξάνεται εκθετικά — το όριο είναι 0. ■

14.2 Πολυωνυμικό κυριαρχεί πάνω σε πολυλογαριθμικό

Μια συνάρτηση f είναι **πολυλογαριθμικά φραγμένη** αν υπάρχει σταθερά c με $f(n) \leq \log^c n$.

Κάθε πολυωνυμική συνάρτηση αυξάνεται γρηγορότερα από κάθε πολυλογαριθμική.

Τυπικά, για $a > 0$, $b > 0$:

$$\lim_{n \rightarrow \infty} \frac{\log^b n}{n^a} = 0 \quad \Leftrightarrow \quad \log^b n \in o(n^a)$$

Παράδειγμα

Απόδειξη (περίληψη). Θέλουμε να δείξουμε ότι υπάρχει n_0 τέτοιο ώστε για κάθε $n \geq n_0$:

$$(\ln n)^b \leq n^a \iff b \ln \ln n \leq a \ln n \iff \ln \ln n \leq \frac{a}{b} \ln n$$

Με L'Hôpital (χρησιμοποιώντας ότι $(\ln x)' = 1/x$ και τον κανόνα αλυσίδας):

$$\lim_{x \rightarrow \infty} \frac{\ln \ln x}{\frac{a}{b} \ln x} = \frac{b}{a} \lim_{x \rightarrow \infty} \frac{1/(x \ln x)}{1/x} = \frac{b}{a} \lim_{x \rightarrow \infty} \frac{1}{\ln x} = 0$$

Άρα ο λόγος τείνει στο 0, που σημαίνει ότι ο παρονομαστής τελικά «νικάει». ■

Κλειδί

Συνδυάζοντας τα δύο αποτελέσματα:

$$\log^c n \prec n^\epsilon \prec n^c \prec a^n \prec n!$$

για οποιεσδήποτε σταθερές $c > 0$, $\epsilon > 0$, $a > 1$. Αυτή είναι η «βασική ιεραρχία» που χρησιμοποιούμε διαρκώς.

15 Η έννοια του βέλτιστου αλγορίθμου

Για ένα πρόβλημα Π ζητάμε τον **καλύτερο** αλγόριθμο που το επιλύει — αυτόν με την καλύτερη πολυπλοκότητα.

Πιο τυπικά: αν κάθε αλγόριθμος που επιλύει το Π έχει ασυμπτωτική πολυπλοκότητα $\Omega(f(n))$ — αυτό είναι **κάτω φράγμα του προβλήματος** — τότε ένας αλγόριθμος με πολυπλοκότητα $O(f(n))$ είναι **(ασυμπτωτικά) βέλτιστος**: ακουμπάει το θεωρητικό όριο.

Διαίσθηση

Παράδειγμα. Για την ταξινόμηση συγκρίσεων, αποδεικνύεται ότι κάθε αλγόριθμος χρειάζεται $\Omega(n \log n)$ συγκρίσεις στη χειρίστη περίπτωση (κάτω φράγμα). Η συγχωνευτική ταξινόμηση τρέχει σε $O(n \log n)$ — άρα είναι **ασυμπτωτικά βέλτιστη**. Θα το αποδείξουμε στο επόμενο μάθημα, για το Divide & Conquer.

16 Κλείσιμο: πολλαπλασιασμός τετραγωνικών πινάκων

Ένα εξαιρετικό παράδειγμα του γιατί η ασυμπτωτική ανάλυση είναι **ζωντανό** ερευνητικό αντικείμενο.

Παράδειγμα

Πρόβλημα. Δοθέντων δύο $n \times n$ πινάκων $A = (a_{ij})$, $B = (b_{ij})$, υπολόγισε τον πίνακα $C = A \cdot B$, όπου:

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$$

Naive αλγόριθμος.


```

Για i = 1 έως n:
  Για j = 1 έως n:
    c_ij <- 0
    Για k = 1 έως n:
      c_ij <- c_ij + a_ik * b_kj

```

Ανάλυση. Τρεις φωλιασμένοι βρόχοι μέχρι το $n \rightarrow n^3$ πολλαπλασιασμοί $\rightarrow \Theta(n^3)$.

Κάτω φράγμα. Χρειαζόμαστε να υπολογίσουμε n^2 στοιχεία του C . Άρα κάθε αλγόριθμος είναι τουλάχιστον $\Omega(n^2)$. **Δεν γνωρίζουμε αλγόριθμο που να επιτυγχάνει $O(n^2)$.**

Το **χάσμα** $[\Omega(n^2), O(n^3)]$ είναι από τα πιο διάσημα ανοιχτά προβλήματα. Η ιστορία του:

Έτος	Ερευνητές	Πολυπλοκότητα
1969	Strassen	$O(n^{2.81})$
1986	Coppersmith & Winograd	$O(n^{2.379})$
2011	V. V. Williams	$O(n^{2.373})$
2023	Williams, Xu, Xu, Zhou	$O(n^{2.371552})$
2025	Alman, Duan, Williams, Xu, Xu, Zhou	$O(n^{2.371339})$

Κάθε δεκαετία μικρο-βελτιώσεις στον εκθέτη. Δεν ξέρουμε αν το $\Omega(n^2)$ είναι το πραγματικό κάτω φράγμα — αλλά **πολλοί ερευνητές πιστεύουν ότι είναι** και ότι κάποτε θα φτάσουμε εκεί.

Ανακεφαλαίωση

Τι κρατάμε από αυτή τη διάλεξη

1. O, Ω, Θ : άνω, κάτω, και ακριβές ασυμπτωτικό φράγμα. Ο ορισμός βασίζεται σε σταθερά c και αρχικό n_0 .
2. **Σταθερές δεν μετράνε:** $O(100n) = O(n)$. Μόνο ο **κυρίαρχος όρος** μετράει.
3. **Πολυωνυμικό vs εκθετικό:** η μεγάλη διαχωριστική γραμμή. Ταχύτερος υπολογιστής δεν σώζει εκθετικό αλγόριθμο.
4. **Ιεραρχία:** $1 \prec \log n \prec n \prec n \log n \prec n^2 \prec \dots \prec 2^n \prec n!$.
5. **Όρια ως εργαλείο:** $\lim f/g = 0 \rightarrow f \in o(g)$, $\lim = c \rightarrow f \in \Theta(g)$, $\lim = \infty \rightarrow f \in \omega(g)$.
6. **L'Hôpital:** λύνει σχεδόν όλα τα όρια που χρειαζόμαστε.
7. **Βέλτιστος αλγόριθμος:** αν το κάτω φράγμα του προβλήματος είναι $\Omega(f)$ και υπάρχει αλγόριθμος $O(f)$, είναι ασυμπτωτικά βέλτιστος.